

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Macari Chiril-Adam, FAF-243
Carazanu Augustin, FAF-243

Report

Lab Work No.1

Network Programming

Concurrency in Practice

Checked by:
asist. univ. Cristofor Fistic

Chișinău – 2026

1. Purpose of the work

The goal of this laboratory work is to measure how the number of threads, and the way the work is divided among them, change the running time and the memory of a real workload: counting the negative, positive and zero numbers among 50 000 000 integers. From our own measurements we decide when the work is CPU-bound and when it is I/O-bound, we observe a race condition and the cost of fixing it, and we find where the parallel speed-up stops.

The work was done in a team of two: **Macari Chiril-Adam** and **Carazanu Augustin** (FAF-243). The source code, the run scripts and the raw results are in the repository https://github.com/chiriladam/network-programming-labs (folder `lab1-concurrency`).

2. Hardware and software

All measurements were taken on the same laptop, connected to the charger, in the Windows “Best performance” power mode and with other programs closed.

Table 1 – Test machine

Component	Value
CPU	AMD Ryzen 5 5600H with Radeon Graphics, 3.3 GHz base clock
Physical cores / logical processors	6 / 12 (SMT: 2 hardware threads per core)
RAM	16 GB (15.4 GB usable), 2 × 8 GB DDR4-3200
Disk	Intel SSDPEKNU512GZ, 512 GB NVMe SSD
USB drive	USB flash drive, USB 3.x (measured read speed ≈ 136 MB/s)
Operating system	Windows 11 Pro, version 10.0.26200, 64-bit
Language and runtime	Java – OpenJDK 25.0.1 (HotSpot 64-Bit Server VM), started with -Xmx2g

3. Implementation

3.1 Program and data

The whole lab is one Java program, `Lab1`. The first command-line argument selects the task, and the options select the configuration, for example `java -cp out Lab1 1a --threads 8` or `java -cp out Lab1 2 --threads 4 --dir E:\parts`. Every task uses only threads created by the program (`java.lang.Thread`, which are OS threads).

`Lab1 gen` writes `numbers.txt`: 50 000 000 random integers from −1 000 000 to 1 000 000 (`SplittableRandom` with seed 42), one per line, 369 445 634 bytes. While generating, it also writes the true counts to `numbers.txt.counts`: **25 001 107 negative, 24 998 868 positive and 25 zero** numbers. `Lab1 split` splits the file, in order, into `part1.txt` … `part8.txt` with 6 250 000 lines each (about 46 MB per file).

3.2 How the measurements are taken

- **Correctness.** At the end of every run the program compares its counts with `numbers.txt.counts` and prints CORRECT or WRONG. The three counts always add up to 50 000 000 in every correct run.

- **Time.** `System.nanoTime()` around the task only: creating the threads, reading, parsing, counting and `join()`. JVM start-up is not included. In Task 1b loading the array is not measured.
- **Peak memory.** The peak working set of the process, which is the most physical memory the process ever used, as reported by Windows (`PeakWorkingSet64`). We also record the peak of the Java heap reported by the JVM. Every configuration runs in a new JVM, so each peak belongs to one configuration only.
- **Repetitions.** In Task 1 every configuration was run 3 times, and the tables and charts use the median. Before the measured runs, `numbers.txt` was read once (not recorded), so all Task 1 runs read it from the OS file cache under the same conditions. In Task 2 each configuration was run once, as the assignment allows.

3.3 Task 1a – splitting the file without losing or repeating lines

The file is divided into as many byte ranges as there are threads: thread *i* gets [size·i/T, size·(i+1)/T). Each thread opens the file itself (`FileChannel`), reads only its own range with a 1 MiB buffer and parses the bytes directly (no `String` objects). A range boundary usually falls in the middle of a line, so we use the rule: **a line belongs to the part in which its first byte lies**. `align(p)` returns the start of the first line at or after byte *p*, and thread *i* processes exactly [align(start), align(end)):

```
static long align(FileChannel ch, long size, long p) throws IOException {
    if (p <= 0) return 0;
    if (p >= size) return size;
    // look at byte p-1: if it is '\n', a line starts exactly at p;
    // otherwise skip to the byte after the next '\n'
    ... scan from p-1 for the first '\n' and return its position + 1
}
```

The end of part *i* and the start of part *i+1* are the same byte offset, and both threads compute `align` of that same offset, so one part ends exactly where the next begins. The last line of a part is read to its end even if it goes past the raw boundary. We tested this with files of 1 to 1000 lines, 1 to 64 threads (more threads than lines too), `\n` and `\r\n` line endings and a file without a final newline (480 combinations). Every run was correct.

3.4 Task 1b – three ways to count

The numbers are first loaded into an `int[50 000 000]` (not measured). Then 12 threads (the number of logical processors) each count an equal slice of the array:

```
// version 1: shared counters, no synchronisation (plain long fields)
if (v < 0) s.negative++; else if (v > 0) s.positive++; else s.zero++;

// version 2: the same shared counters, each increment under one ReentrantLock
lock.lock();
try { if (v < 0) s.negative++; else if (v > 0) s.positive++; else s.zero++; }
finally { lock.unlock(); }

// version 3: local variables of the thread, added by main after join()
if (v < 0) neg++; else if (v > 0) pos++; else zero++;
```

3.5 Task 1c – producer and consumers

One reader thread reads `numbers.txt` with `BufferedReader.readLine()` and puts the lines, in batches of 10 000, into an `ArrayBlockingQueue` with room for 100 batches. `put()` blocks while the

queue is full and `take()` blocks while it is empty. At the end of the file the reader closes the queue by putting one special END batch (a poison pill) per worker. The queue is FIFO, so the END batches come after all real batches, and a worker stops when it takes one. Each worker counts into its own `Counts` object, and the main thread adds them up after `join()`.

```
while (true) {
    List<String> batch = queue.take();   // waits while the queue is empty
    if (batch == END) break;             // queue closed and empty
    for (String s : batch) mine.add(Integer.parseInt(s.trim()));
}
```

3.6 Task 2 – many files on slow storage

The 8 part files are the units of work. They are dealt to the threads round-robin (file *k* goes to thread *k* mod T), so 1 thread gets 8 files, 2 threads get 4 each and 4 threads get 2 each. A thread reads a whole file with one `Files.readAllBytes()` call into one `byte[]`, then parses that buffer. The files were copied to the USB drive, and before every run the drive was ejected, unplugged and plugged back in, so no data came from the file cache. For comparison, the same 8 files were also read from the NVMe SSD (with a warm cache).

4. Task 1a: counting with threads

Table 2 – Task 1a results (median of 3 runs; all runs CORRECT)

Threads	Time (ms)	Speed-up	Peak memory, process (MB)	Peak Java heap (MB)
1	1084	1.00	51	6
2	622	1.74	53	8
4	352	3.08	58	13
8	260	4.17	66	23
16	225	4.83	83	43
32	252	4.31	97	83
64	252	4.30	125	143

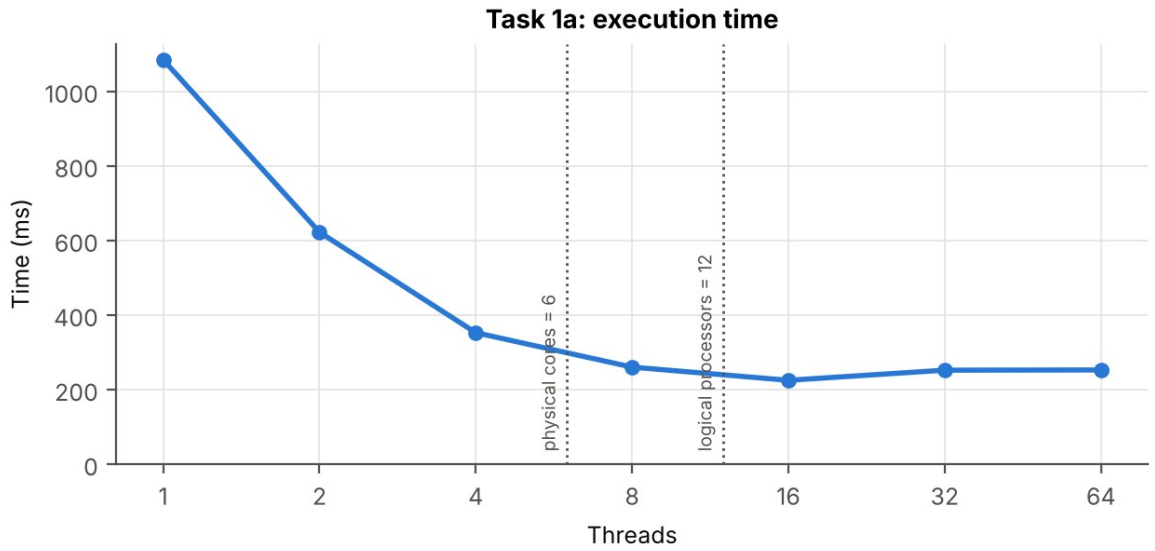


Figure 1 – Task 1a: execution time against the number of threads

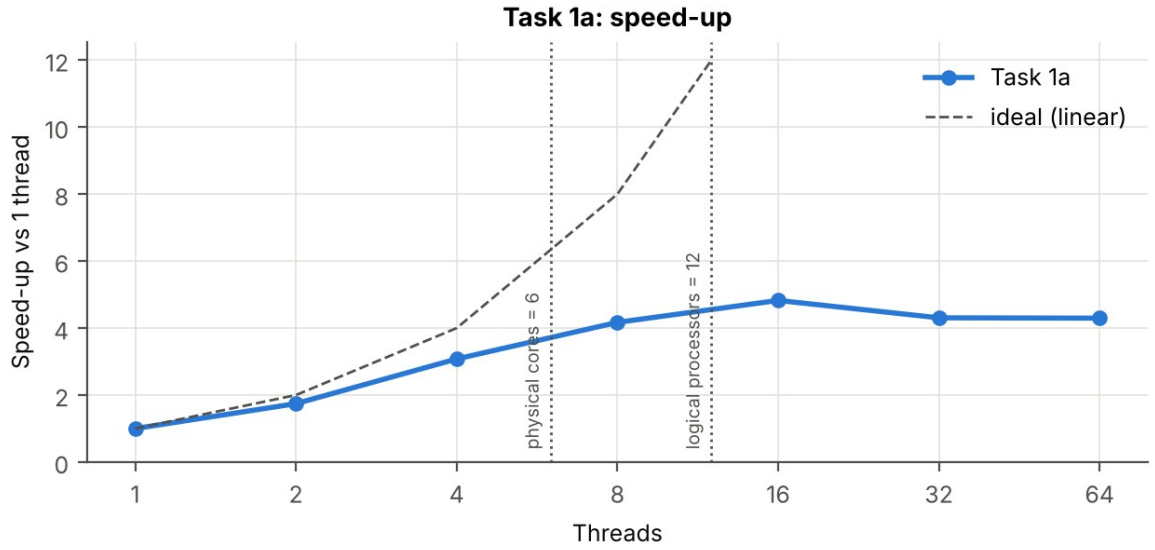


Figure 2 – Task 1a: speed-up relative to one thread (dashed line: ideal linear speed-up)

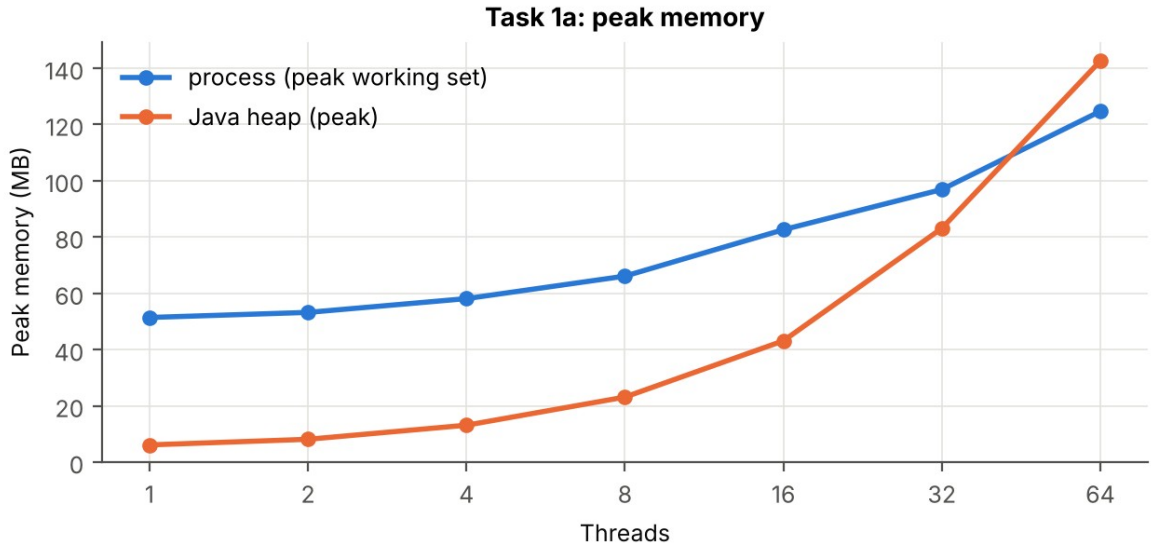


Figure 3 – Task 1a: peak memory against the number of threads

Answers to the questions of Task 1

Q1. ***At which number of threads does the speed-up stop growing? Relate it to the physical cores and logical processors.***

The speed-up grows almost linearly up to 4 threads (3.08×). At 8 threads, just above the **6 physical cores**, it reaches 4.17×. Between 8 and 16 threads it grows only 16% more, to the maximum of **4.83× at 16 threads**, after which it stops. So the strong growth ends at the number of physical cores. The extra gain up to the **12 logical processors** comes from SMT: two hardware threads share one core and fill each other's idle execution units, which gives some percent, not twice the speed. Above 12 there is no more hardware to run on. The speed-up stays below the ideal 6× because the laptop lowers its boost clock when all cores are busy, and the threads share the memory bandwidth and the L3 cache.

Q2. ***Why do 32 and 64 threads not help? Are they slower, and why?***

They do not help because the processor can run only 12 threads at the same moment, and the work is pure computation with no waiting. With 32 or 64 threads the extra threads just wait their turn. They are

slightly slower: **252 ms instead of 225 ms** (about 12%). The total work is the same, but there is more overhead: creating and joining more threads, more context switches as the OS scheduler time-slices them, each thread opening its own file channel and buffer, and more threads competing for the same caches.

Q3. ***Does the peak memory depend on the number of threads? Why?***

Yes, it grows linearly, from **51 MB with 1 thread to 125 MB with 64 threads**, about 1.2 MB per extra thread. The numbers themselves are never all in memory: every thread streams its part through its own 1 MiB read buffer. So the memory is the fixed JVM part (about 50 MB) plus, for each thread, its read buffer and its stack. The Java heap peak is an upper bound: it adds the peaks of the separate heap regions, which do not happen at the same moment. That is why at 64 threads it can be higher than the process peak measured by Windows.

One run with 16 threads took 661 ms instead of about 225 ms, because of a background process. This is why we use the median of 3 runs.

5. Task 1b: three ways to count

Table 3 – Task 1b results, 12 threads (expected: 25 001 107 / 24 998 868 / 25)

Version	Run	Time (ms)	Negative	Positive	Zero	Total	Result
1: no sync	1	546	4 886 641	5 070 349	25	9 957 015	WRONG
1: no sync	2	547	4 584 798	4 752 157	25	9 336 980	WRONG
1: no sync	3	542	4 445 242	4 550 741	25	8 996 008	WRONG
2: lock	median	1066	25 001 107	24 998 868	25	50 000 000	CORRECT
3: per-thread	median	111	25 001 107	24 998 868	25	50 000 000	CORRECT

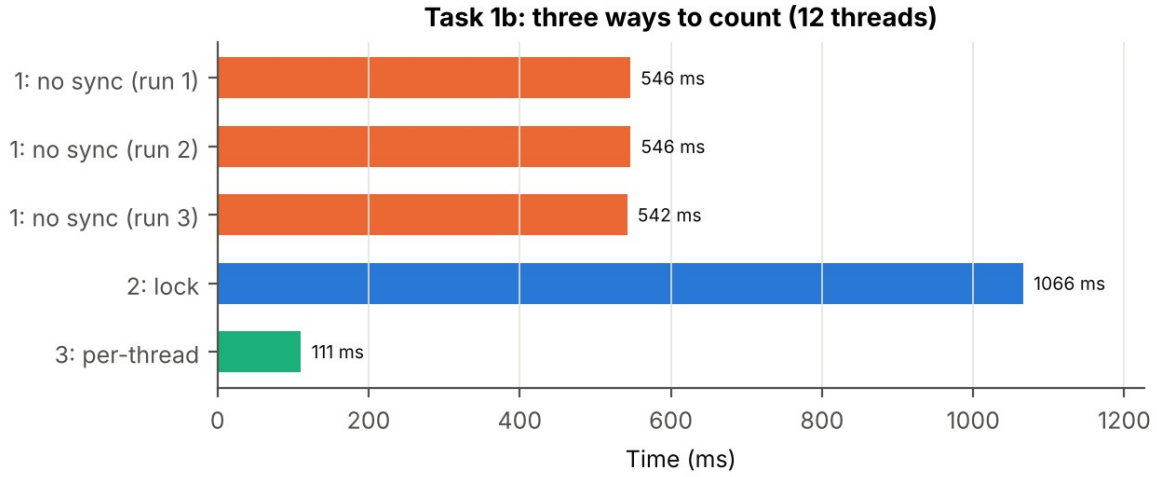


Figure 4 – Task 1b: execution time of the three versions (12 threads)

Peak memory was the same for the three versions, about 250 MB. It is dominated by the 200 MB array, which all versions share.

Answer to question 4

Q4. ***Rank the three versions of Task 1b by time. Which of them are correct? Explain each result.***

From fastest to slowest: **version 3 (per-thread counters) ≈ 110 ms**, **version 1 (no synchronisation) ≈ 545 ms**, **version 2 (lock) ≈ 1066 ms**. Only versions 2 and 3 are correct.

- **Version 1 is wrong.** In the three runs it counted only 9.96, 9.34 and 9.00 million numbers instead of 50 million, so 80–82% of the increments were lost, and the result changes from run to run. `counter++` is three steps (load, add 1, store). When two threads load the same value at the same time, both store value+1 and one increment disappears. The zero counter happened to be right (25), because there are only 25 zeros and two threads almost never update it at the same moment. Version 1 is also 5× slower than version 3, although it has no lock: the three counters are next to each other in one object, so they are in the same 64-byte cache line. Every write needs exclusive ownership of that line, so the line keeps moving between the 12 cores' caches (cache-line ping-pong).
- **Version 2 is correct but the slowest.** The lock makes every increment atomic, but it is taken and released 50 million times. Each acquisition is an atomic compare-and-swap with memory barriers, and under contention threads also have to wait and be woken up. Only one thread at a time can count, so the 12 threads effectively work one after another, plus the cost of the lock.
- **Version 3 is correct and the fastest.** Each thread increments its own local variables, which stay in CPU registers. Nothing is shared, so there is no lock, no cache-line traffic and no lost update. The main thread adds the 12 partial results only once, after `join()`, which is safe because `join()` guarantees that the threads' writes are visible.

6. Task 1c: producer and consumers

Table 4 – Task 1c results, batches of 10 000 lines, queue of 100 batches (median of 3 runs; all CORRECT)

Workers	Time (ms)	Speed-up	Peak memory, process (MB)	Peak Java heap (MB)
1	2294	1.00	214	155
2	2237	1.03	213	150
4	2286	1.00	210	150
8	2366	0.97	210	150
16	2421	0.95	211	150
32	2423	0.95	210	150
64	2426	0.95	214	150

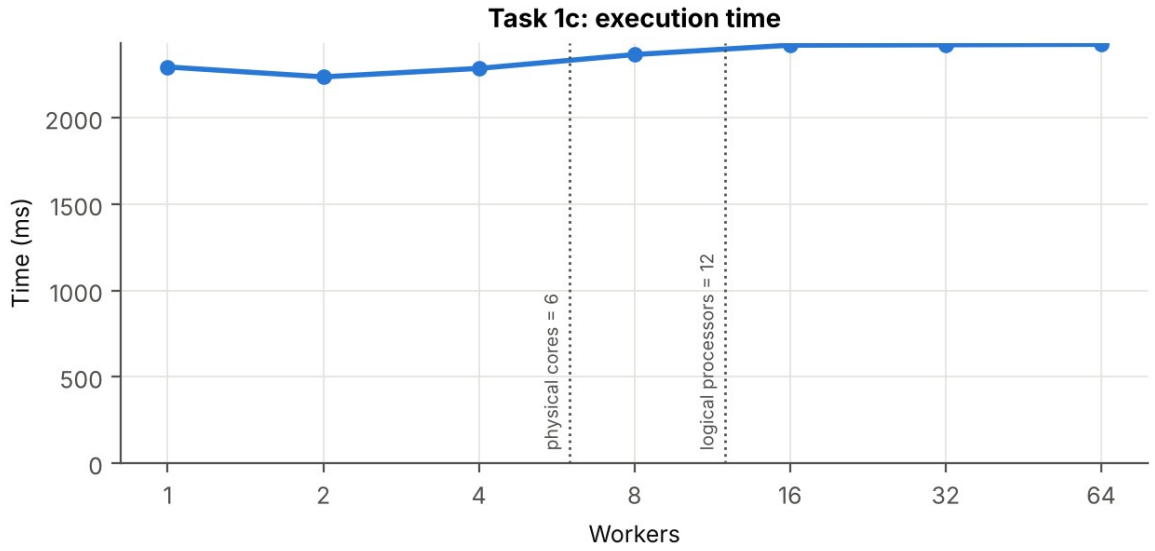


Figure 5 – Task 1c: execution time against the number of workers

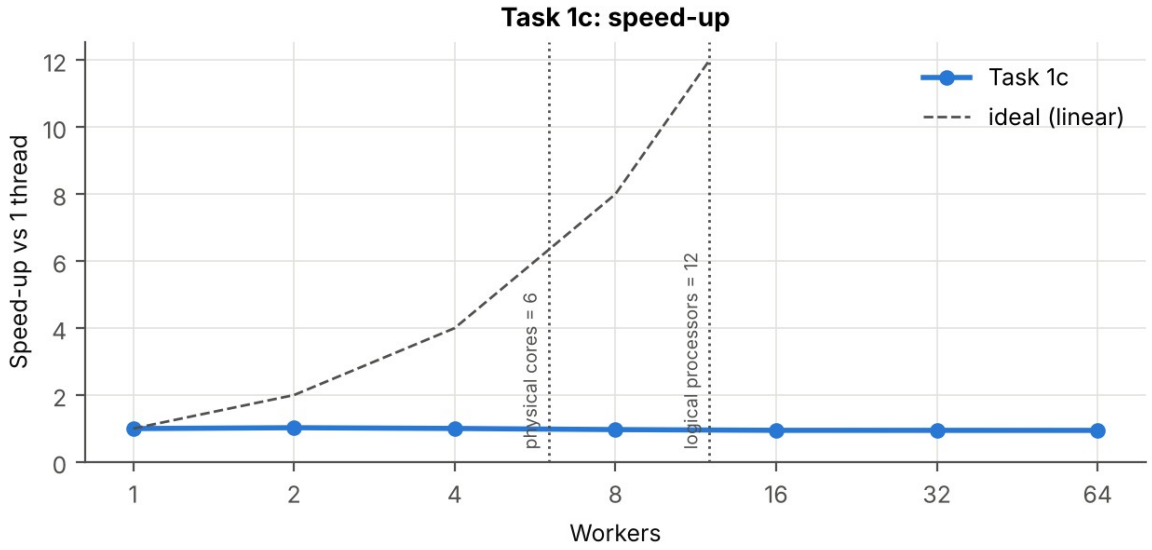


Figure 6 – Task 1c: speed-up relative to one worker

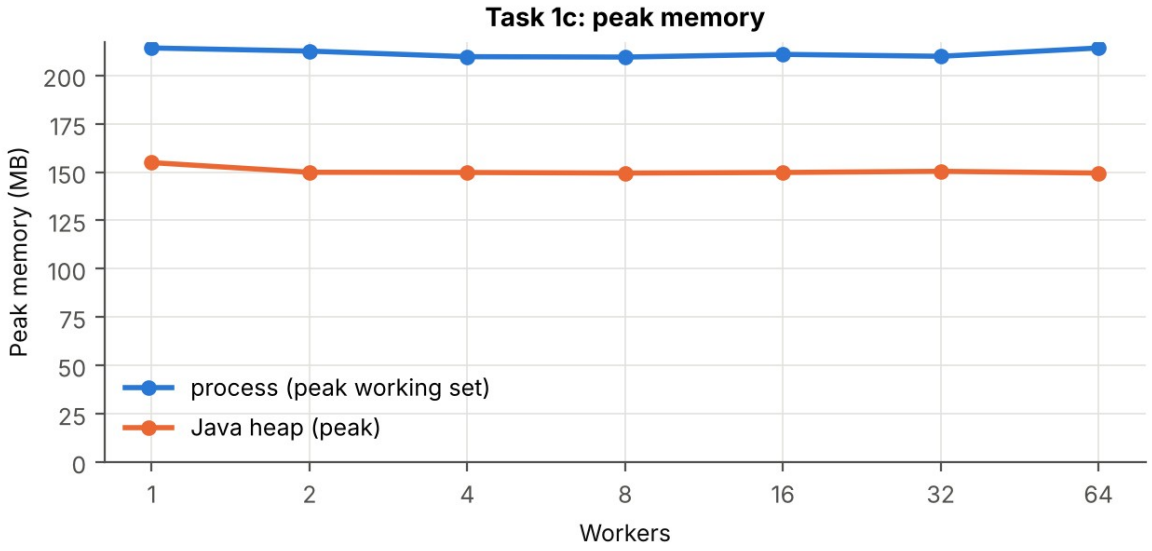


Figure 7 – Task 1c: peak memory against the number of workers

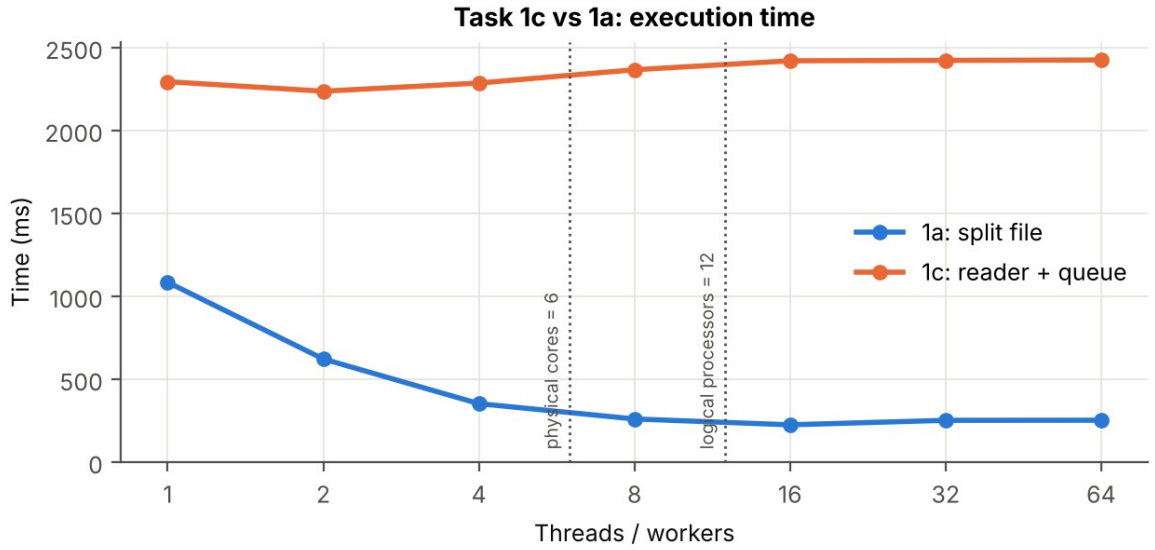


Figure 8 – Task 1c compared with Task 1a

Table 5 – Extra experiments for question 6 (one run each, all CORRECT)

Configuration	Workers	Time (ms)	Peak memory, process (MB)
Batches of 1 line, queue of 100	4	27 709	202
Batches of 10 000, queue without size limit	1	2 218	212
Batches of 10 000, queue without size limit	4	2 354	209
Reference: batches of 10 000, queue of 100	4	2 286	210

Answers to questions 5 and 6

Q5. *Compare Task 1c with Task 1a. Which one is faster, and why? At which number of workers does Task 1c stop getting faster?*

Task 1a is much faster: 1084 ms against 2294 ms on one thread, and 225 ms against 2237 ms at their best, about 10× faster. Task 1c **does not get faster at all**. The best time is with 2 workers (2237 ms), only 3% better than 1 worker, and from 8 workers on it is slightly slower.

The reason is the single reader. In Task 1c all the reading is done by one thread, and `readLine()` is expensive: it decodes the bytes into characters, looks for the end of the line and creates a new `String` object for every line (50 million objects that the garbage collector then has to free). This sequential part takes about 2.2 s, and no number of workers can make it shorter. Parsing a batch is faster than reading it, so even one worker keeps up with the reader. The other workers spend almost all their time blocked in `take()` on an empty queue. More workers only add more threads that compete for the queue's lock and have to be woken up, which is why the time rises slightly. In Task 1a the reading itself is parallel: every thread reads and parses its own part, straight from the bytes, without creating objects.

Q6. *What would happen with batches of a single line? And with a queue that has no size limit?*

Batches of one line: we measured it, and with 4 workers the run took **27.7 s instead of 2.3 s**, 12× slower. Every line now costs a `put()` and a `take()`, each of which takes the queue's lock, and often a signal that wakes a sleeping thread (a system call). It also creates one list object per line. This synchronisation cost is much larger than the work of parsing one number, so the threads spend their time coordinating instead of counting. Batching spreads one synchronisation over 10 000 numbers.

A queue with no size limit: in our measurement nothing changed (2.2–2.4 s, about 210 MB), because the workers are faster than the reader, so the queue is always almost empty and the limit of 100 batches is never reached. The danger appears when the consumers are slower than the producer (slower processing, fewer workers, a slow network). Then nothing stops the reader: the queue grows until it holds the whole file. That is 50 million `String` objects, about 50–60 bytes each, so roughly 2.5–3 GB, more than the 2 GB heap, and the program would fail with `OutOfMemoryError`. With the limit, a full queue blocks the reader (back-pressure), so memory stays below 100 × 10 000 lines, about 50 MB, whatever the speeds are.

7. Task 2: many files on slow storage

Table 6 – Task 2 results: the same 8 files (369 MB) read from the USB drive and from the SSD; all CORRECT

Threads	USB time (ms)	USB speed-up	USB peak memory (MB)	SSD time (ms)	SSD speed-up
1	3785	1.00	183	1091	1.00
2	2820	1.34	227	639	1.71
4	2711	1.40	409	456	2.39

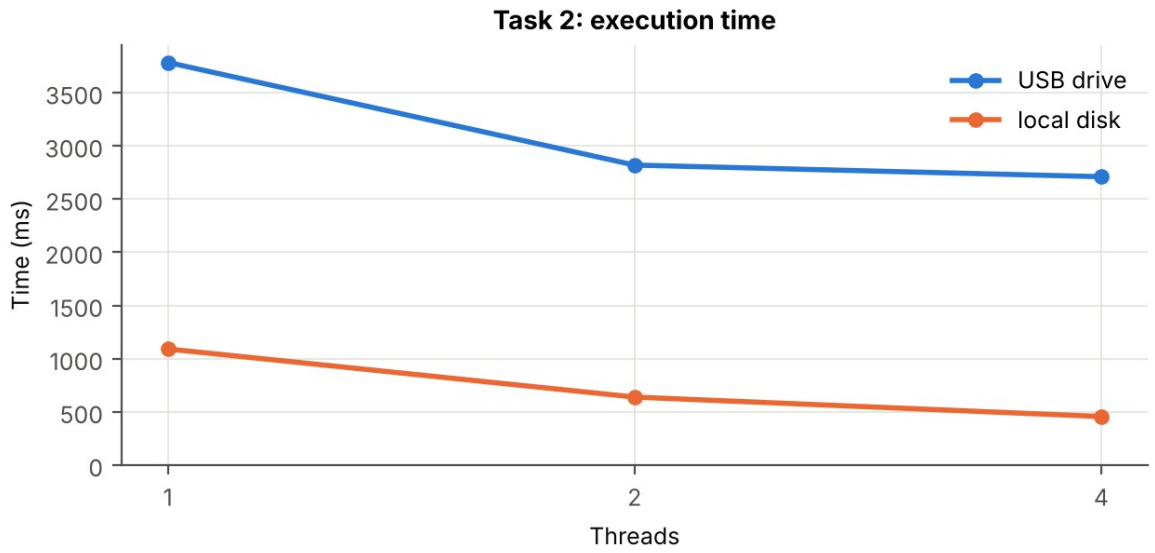


Figure 9 – Task 2: execution time, USB drive and local SSD

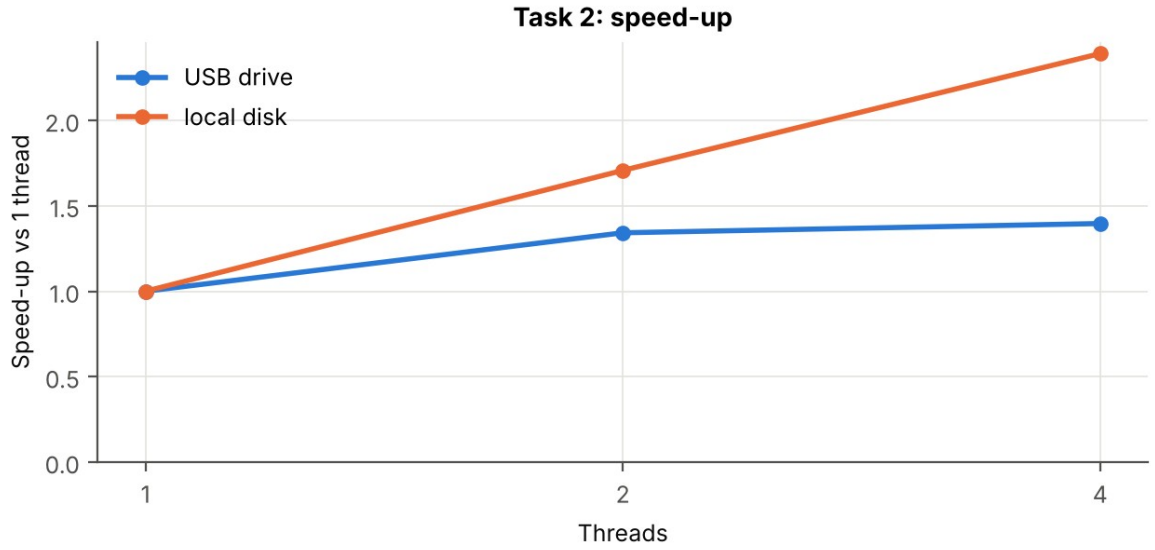


Figure 10 – Task 2: speed-up relative to one thread

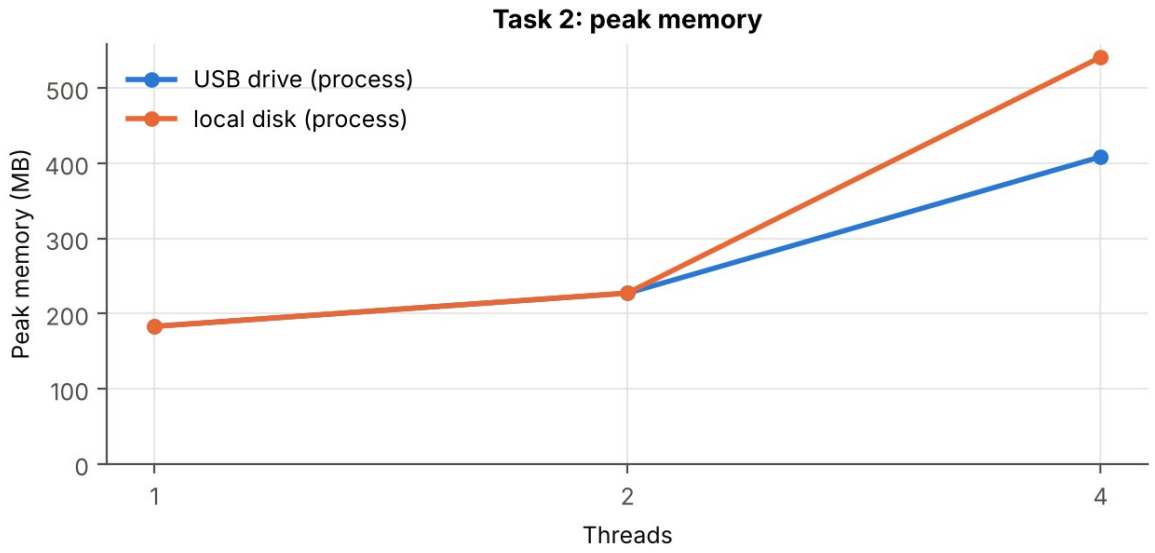


Figure 11 – Task 2: peak memory against the number of threads

Peak memory grows with the threads (183 → 227 → 409 MB on USB), because every thread holds one whole file, about 46 MB, in a `byte[]`. With 4 threads four such buffers are in memory at the same time, plus the buffers of already processed files that the garbage collector has not freed yet.

Answers to the questions of Task 2

Q1. *Compare the time on one thread with Task 1a on one thread. On the USB drive, is the work CPU-bound or I/O-bound?*

On one thread, Task 1a (same numbers, from the SSD and the file cache) takes **1084 ms**, and Task 2 from the SSD takes **1091 ms**. So parsing the 50 million numbers costs about **1.1 s of CPU time**. The same work from the USB drive takes **3785 ms**, 3.5× longer. The extra **≈2.7 s, about 71% of the run**, is spent waiting for the drive, while the CPU is idle. **The work is I/O-bound on the USB drive.** The time is set by how fast the drive can deliver 369 MB, not by how fast we parse. With 4 threads the whole run takes 2.71 s, which corresponds to 369 MB / 2.71 s ≈ **136 MB/s**: the read speed of the drive itself (well above the ~40 MB/s limit of USB 2.0, so the drive works in USB 3.x mode).

Q2. ***On the USB drive, do more threads help? Why not, when lecture 1 showed that I/O-bound work benefits from concurrency?***

They help only a little, once: 2 threads are 1.34× faster than 1, and 4 threads (1.40×) are almost the same as 2. With one thread, reading and parsing alternate: while the file is read the CPU waits, and while it is parsed the drive waits, so the time is about read + parse ≈ 2.7 + 1.1 s. With two threads one thread parses while the other reads, so the parsing hides behind the reading and the time drops to about the read time alone (2.8 s). After that, more threads cannot help: all of them read from **the same single device**, whose bandwidth (≈136 MB/s) is already fully used. Parallel requests only split that bandwidth between them. For cheap flash memory, interleaved requests can even be slower than sequential ones.

The example in lecture 1 is different: there the threads wait on **latency** of independent resources, for example many requests to servers over the network. While a request waits for an answer, it uses almost no local resource, so many requests can wait at the same time and the waits overlap. With the USB drive the bottleneck is not waiting time but the **throughput** of one shared device. That resource is already saturated, and concurrency cannot create more of it.

8. Conclusion

On our machine (Ryzen 5 5600H, 6 cores / 12 logical processors, NVMe SSD), counting the numbers of a file that is already in the file cache is **CPU-bound**. In Task 1a the time fell from 1084 ms to 225 ms as we added threads, the speed-up followed the number of cores (strong up to 6, a little more up to 12 logical processors) and stopped there. 32 and 64 threads were slightly slower. The same holds for counting an array in memory (Task 1b). There we also saw that sharing mutable counters is wrong without synchronisation (80% of the updates lost) and slow with a lock (1066 ms). Per-thread counters merged at the end were both correct and 10× faster than the lock.

Task 1c showed that a parallel design is only as fast as its sequential part: with one reader thread the program stayed at about 2.3 s for any number of workers. Batching was essential: batches of one line were 12× slower. The bounded queue protects the memory if the consumers fall behind.

Reading the files from the USB flash drive was **I/O-bound**. The same parsing that needs 1.1 s of CPU took 3.8 s from the drive, the CPU was idle 71% of the time, and extra threads gave at most 1.4×, limited by the drive's ≈136 MB/s. We know which case is which from two measurements: how the time changes when the same work is read from fast or slow storage, and how it changes when we add threads. CPU-bound work speeds up with threads up to the number of cores. I/O-bound work on one device stays at the device's limit.

9. Answers to the control questions

1. ***What is the difference between a physical core and a logical processor? Where did it show in your charts?***

A physical core is a complete execution unit: its own pipelines, ALUs, FPU and L1/L2 caches. A logical processor is what the OS sees as a CPU. With SMT (Simultaneous Multi-Threading) one physical core runs two hardware threads, which share its execution units and caches. They take turns using the parts the other thread leaves idle, so two logical processors on one core give much less than twice the speed. Our CPU has 6 cores and 12 logical processors. In Figure 2 the speed-up rises steeply

up to about 6 threads, rises only a little more up to 12–16 threads (the SMT gain), and stops after the 12 logical processors.

2. ***How did you make sure that no line was counted twice or skipped at the boundary between two parts of the file?***

A line belongs to the part in which its first byte lies. Each thread moves both of its raw boundaries forward to the start of the next line (`align`). If the byte before the boundary is '\n', the boundary is already a line start; otherwise it skips to the byte after the next '\n'. Thread *i* processes [align(start), align(end)) and finishes the line that crosses its end. The end of part *i* and the start of part *i+1* are the same offset and both threads compute `align` of it in the same way, so the parts touch exactly, with no gap and no overlap. We checked it by comparing every result with the true counts and by testing 480 small cases (1–1000 lines, 1–64 threads, LF/CRLF, with and without a final newline).

3. ***Explain step by step how unsynchronised shared counters lose updates.***

`negative++` is not one operation but three: (1) load the value from memory into a register, (2) add 1 in the register, (3) store it back. Suppose `negative` is 100. Thread A loads 100. Before A stores, thread B on another core also loads 100. A adds 1 and stores 101. B adds 1 and stores 101. Two numbers were counted, but the counter grew by one: A's update is lost. With 12 threads incrementing the same three counters millions of times, this happens all the time. In our runs 80–82% of the increments were lost, and every run gave a different total.

4. ***Why are per-thread counters faster than shared counters protected by a lock?***

With a lock, every one of the 50 million increments has to acquire and release the lock. That is an atomic compare-and-swap plus memory barriers, and when the lock is busy the thread waits or is put to sleep and woken up later. Only one thread counts at a time, and the lock's state and the counters keep moving between the cores' caches. Per-thread counters share nothing: each thread increments variables that stay in its own registers at full speed, and all threads really work in parallel. Synchronisation happens only once at the end (`join()` and 12 additions). In our measurement: 110 ms against 1066 ms.

5. ***What does "I/O-bound" mean for a USB drive, and for a server waiting on the network? Why does concurrency help in only one of the two cases?***

I/O-bound means that the program spends most of its time waiting for input/output, not computing. For the USB drive, the program waits because one device delivers data at a limited speed (≈136 MB/s). The bottleneck is the **throughput** of a single shared resource, and once it is fully used, more threads only divide it. For a server waiting on the network, the program waits mostly because of **latency**: the request travels, the remote server works, the answer comes back. During that wait almost no local resource is used. With many concurrent requests, their waits overlap, so ten requests take about as long as one. That is why concurrency helps in the network case and not with the single USB drive.

6. ***What is the difference between dividing the work into equal parts in advance and taking it from a queue? When is each one better?***

Equal parts in advance (static partitioning, Task 1a, 1b, 2) need no coordination while working: each thread knows its part from the start, so there is no shared queue and no locking. It is best when the work can be split cheaply and fairly in advance (random access to the data, equal cost per item). But if one part is slower (a slow file, an unlucky core), the others finish and wait. Taking work from a queue

(dynamic distribution, Task 1c) balances the load automatically: a thread that finishes early simply takes the next batch. It works even when the data arrives as a stream that only one thread can read, or when the total amount of work is not known in advance. The price is synchronisation on every take, which is why work must be taken in batches (Q6), and the producer can become the bottleneck (Q5).

10. Source code and how to run

Repository: https://github.com/chiriladam/network-programming-labs (folder `lab1-concurrency`). Requirements: JDK 17 or newer (we used OpenJDK 25); for the charts, Python 3 with matplotlib.

Table 7 – Commands (run in the project folder)

What	Command
Build	`javac -d out src/*.java` (or `scripts\build.ps1`)
Generate the data	`java -cp out Lab1 gen`
Split into 8 files	`java -cp out Lab1 split`
Task 1a	`java -cp out Lab1 1a --threads N`
Task 1b	`java -cp out Lab1 1b --version 1|2|3`
Task 1c	`java -cp out Lab1 1c --workers N [--batch B] [--capacity C]`
Task 2	`java -cp out Lab1 2 --threads N --dir E:\parts`
All of Task 1	`scripts\run_task1.ps1 -Repeats 3 -Extras`
All of Task 2	`scripts\run_task2.ps1 -Drive E: -Copy -AlsoLocal`
Charts	`python scripts/make_charts.py results.csv --physical-cores 6`

Every run prints the counts, CORRECT/WRONG, the time and the peak memory, and appends a line to `results.csv`. The README in the repository describes every option.